

Pretty-printing & Automatic Styling

Topics: strings, string functions, whitespace
Lab 7: https://maryash.github.io/135/labs/lab_07.html

Ryan Vaz

Auto-formatter

In this lab, the goal is to make a program that will take in a `.cpp` file that is badly formatted and apply the correct formatting.

```
int main(){  
    // Hi, I'm a program!  
  
int x = 1;  
    for(int i = 0; i < 10; i++) {  
cout << i;  
        cout << endl;  
    }  
}
```



```
int main(){  
    // Hi, I'm a program!  
    int x = 1;  
    for(int i = 0; i < 10; i++) {  
        cout << i;  
        cout << endl;  
    }  
}
```

Removing the indents

The first step in our program is to remove all the leading indents from each line. We will write a function to do that for us.

```
    int main(){  
        // Hi, I'm a program!  
  
    int x = 1;  
        for(int i = 0; i < 10; i++) {  
    cout << i;  
            cout << endl;  
        }  
    }
```



```
$ ./unindent < bad-code.cpp  
int main(){  
    // Hi, I'm a program!  
  
int x = 1;  
for(int i = 0; i < 10; i++) {  
cout << i;  
    cout << endl;  
}  
}
```

Input redirection

Fstream allows us to take in input from a file, but cin can do the same thing! When you run your program, if you want to send the contents of a file into the standard input (cin), you can run it as follows:

```
$ ./unindent < bad-code.cpp ←  
int main(){  
    // Hi, I'm a program!  
    int x = 1;  
    for(int i = 0; i < 10; i++) {  
        cout << i;  
        cout << endl;  
    }  
}
```

The first line allows the text of the file bad-code.cpp to get redirected into the program using ``<``.

Now if ``cin``, or ``getline(cin, str)`` is used, instead of waiting for a user input from the terminal, it will grab the contents from the file.

With this current program, the input for the executable ``./unindent`` will be obtained from a file called ``bad-code.cpp``

Removing the leading spaces

Implement the function: `string removeLeadingSpaces(string line);`

This function takes in a unformatted string and removes the leading spaces, and returns that value. For example, calling `removeLeadingSpaces(" Hello")` will return `"Hello"`.

We will now write a main function that continuously takes input via `getline`, and feeds those values into our `removeLeadingSpaces` function and outputs it.

The input will be coming from the text file, you'll probably want to make your own badly indented code to test your program.

Main function pseudocode

For the main function, we'll need to keep taking in the user input via a while loop, and then print the output of calling the `removeLeadingSpaces` function on each input. Here's a bit of pseudocode to help guide you:

```
string removeLeadingSpaces(string line){  
    // Initialize an empty string  
    // Initialize a boolean value (flag) to false  
    // Loop through the characters of the input string  
    |   // Set the value of flag to true if the character is not a whitespace  
    |   // If the value of flag is true, append the current character to your string  
    // Return your string  
}
```

Hint: You can use `'isspace()'` from the `'<ctype.h>'` library to find whether a character is a space

```
int main(){  
    // Initialize a empty string  
    while(/* getting a single line of user input using getline */){  
        // Print the output of calling removeLeadingSpaces on the line  
    }  
}
```

Indenting unindented lines

If we want to correctly indent each line, we need to count how many { and } are there on each line. For this lab, assume that all { increase the number of indents, and all } decreases the number of indents

```
int main(){ // 0 indent (+1)
// Hi, I'm a program! // 1 indent
int x = 1; // 1 indent
for(int i = 0; i<10; i++){ // 1 indent (+1)
cout << i; // 2 indent
cout << endl; // 2 indent
} // 1 indent (-1)
} // 0 indent (-1)
```

Counting curly braces

You should write a function to count both open and closed curly braces:

```
int countChar(string line, char c);
```

This function will count how many times a parameter `c` occurs in the string parameter `line`. It boils down to a for-loop with an if-statement.

Here are some example runs:

```
countChar("{} Why {}", '{') == 3
```

```
countChar("{}Where{}", '}') == 2
```


Updating the main function

```
int main(){  
    // Initialize a empty string  
    // Initialize an integer value (tabs) to 0  
    while(/* getting a single line of user input using getline */){  
        // Print `tabs` number of tab characters  
        // Print the output of calling removeLeadingSpaces on the line  
        // Increment/Decrement the tabs value by the number of '{', '}' in user input  
        // Hint: Does { increment before or after printing the line,  
        //           } decrement before or after printing the line?  
    }  
}
```